CacheScope

Intelligent Cache Behavior Analyzer & Optimizer

Project Specification, Architecture, Concepts, Features, and Implementation Plan

1. Project Overview

CacheScope is a Python-based cache-memory analysis tool. It is designed to do more than simulate whether individual memory accesses hit or miss. The program accepts or generates memory-access traces, simulates cache behavior, classifies misses, visualizes cache usage, detects access patterns, and runs what-if experiments on different cache configurations.

The key idea is that the project should explain WHY a result changed. For example, instead of only reporting that an 8 KB cache achieved a higher hit rate than a 4 KB cache, the analyzer should identify whether the change was associated with fewer capacity misses, fewer conflict misses, better locality, or another configuration change.

2. Main Goal

Build an educational but technically meaningful cache analyzer that connects cache-memory theory with an executable Python simulation.

- Simulate cache accesses from memory-address traces.
- Support direct-mapped, set-associative, and fully associative caches.
- Support multiple replacement policies.
- Calculate hit rate, miss rate, evictions, and AMAT.
- Classify misses such as compulsory, conflict, and capacity misses.
- Visualize set utilization and cache behavior.
- Detect common access patterns such as sequential, random, looping, and locality-heavy behavior.
- Run what-if experiments across cache sizes, block sizes, associativities, and replacement policies.
- Explain the reason for changes rather than simply declaring one configuration 'better'.

3. Core Cache Concepts Used by the Project

3.1 Cache Hit and Cache Miss

A cache hit occurs when the requested memory block is already present in the cache. A cache miss occurs when the requested block is not present, so the processor must obtain it from a slower memory level.

Example:

Cache initially contains [10, 20, 30]. Requests: 10, 20, 40, 10, 50.

Hits = 2 (10 and 20 on their first requests, depending on the stated initial cache). For the simplified example used during discussion, the important formulas are:

Hit Rate = hits / total accesses × 100

Miss Rate = misses / total accesses × 100

3.2 Conflict Miss

A conflict miss occurs when different memory blocks map to the same cache set and compete for the available ways in that set. This can happen even when other sets have unused space.

For a cache with 4 sets:

Block 0 → Set 0
Block 4 → Set 0
Block 8 → Set 0

because:

set_number = block_number % number_of_sets

If Set 0 is 2-way and already contains two blocks, a third block mapped to Set 0 requires an eviction. Increasing associativity can reduce this kind of conflict.

3.3 Capacity Miss

A capacity miss occurs because the cache is too small for the workload's working set. For example, with a cache that can hold only 3 blocks, repeatedly accessing A, B, C, D, A, B, C, D can continually evict blocks because 4 blocks are needed.

Difference:

Capacity miss = insufficient total cache capacity.

Conflict miss = blocks compete for the same set even though total capacity may be sufficient.

3.4 Associativity

Associativity is the number of ways per set: the number of cache lines that can potentially hold blocks mapped to the same set.

Associativity	Meaning
1-way / direct mapped	One cache line per set
2-way	Two lines per set
4-way	Four lines per set
Fully associative	A block can be placed in any cache line

In a set-associative cache, a block maps to one set, but it can occupy any way within that set. The replacement policy determines which way to evict when the set is full.

3.5 Block Size

Block size is the amount of memory stored in one cache block/cache line. For example, a 64-byte block holds 64 bytes. The cache normally fetches the whole block, which exploits spatial locality.

Example: 1 KB cache with 64-byte blocks → 1024 / 64 = 16 cache blocks.

3.6 Mapping a Block to a Set

For a set-associative cache:

set_number = block_number % number_of_sets

Example with 4 sets: blocks 0, 4, 8, 12 map to Set 0; blocks 1, 5, 9, 13 map to Set 1; and so on.

3.7 From Address to Block, Set, Offset, and Tag

Given a byte address and block size:

block_number = address // block_size

offset = address % block_size

set_number = block_number % number_of_sets

tag = block_number // number_of_sets

The tag formula above is a convenient integer form for the cache geometry. In a real hardware address, the tag consists of the higher-order address bits after removing offset and set-index bits.

Example with 64-byte blocks:

Addresses 0–63 → Block 0
64–127 → Block 1
128–191 → Block 2
192–255 → Block 3

Address 100 → Block 1, Offset 36.

Address 200 → Block 3, Offset 8.

3.8 How a Cache Decides Hit or Miss

Conceptual address structure:

TAG | SET INDEX | OFFSET

A cache line can be thought of as containing a valid bit, a tag, and the block data.

VALID | TAG | DATA

For a hit, the selected set must contain a valid line whose stored tag matches the requested tag.

Flow:

1. CPU requests an address.
2. Calculate the block number.
3. Calculate the set.
4. Look inside that set.
5. Compare the requested tag with tags in the set.
6. Matching valid tag → HIT.
7. No matching valid tag → MISS.

4. Important Mapping vs Replacement Rule

A central rule for the project is: mapping determines WHICH SET a block must use; replacement policy determines WHICH WAY inside that set is replaced when the set is full.

Example: if there are 4 sets, Block 5 maps to Set 1 because 5 % 4 = 1.

If Set 1 has an empty way, Block 5 can use that empty way. If Set 1 is full, the replacement policy chooses which existing block/way to evict.

Examples of replacement decisions:

- LRU → evict the least recently used block.
- FIFO → evict the block that arrived first.
- Random → choose a way randomly.
- LFU → evict the least frequently used block.
- Optimal → for analysis/benchmarking, evict the block whose next use is farthest in the future.

5. Proposed Feature Set

Feature	Purpose
Cache simulation	Simulate memory accesses and cache state step by step.
Cache organizations	Direct mapped, set-associative, and fully associative.
Replacement policies	LRU, FIFO, LFU, Random, and Optimal.
Workload generators	Sequential, random, looping, locality-heavy, and matrix-like traces.
Metrics	Hits, misses, hit rate, miss rate, evictions, and AMAT.
Miss classification	Compulsory, conflict, and capacity misses.
Heatmap	Visualize how heavily different sets are being used.
Access-pattern detector	Identify sequential, random, looping, or locality-heavy behavior.
What-if simulator	Compare multiple cache configurations automatically.
Explanation engine	Explain which configuration parameter changed and what cache behavior it affected.

6. What-If Configuration Analysis

The existing cache configuration and the what-if configurations are conceptually different. If the current cache is 4 KB, an 8 KB experiment does not mean the current cache has already become 8 KB. It means the program simulates an alternative cache design to study what would happen.

Example current configuration:

Cache size = 4 KB
Block size = 64 B
Associativity = 1-way

Replacement = Direct mapped
Hit rate = 61%
Conflict misses = 31%

Example automatically generated alternatives:

Configuration	Hit Rate
4 KB, 1-way	61%
4 KB, 2-way	74%
4 KB, 4-way	81%
8 KB, 2-way	86%
8 KB, 4-way	89%

These numbers are illustrative project-output examples, not guaranteed results. Actual hit rates depend on the trace/workload and the cache parameters.

The analyzer should explain the changes. For example, increasing associativity while keeping cache size fixed can reduce conflict misses. Increasing total cache size can reduce capacity misses. Changing block size can change spatial-locality behavior and the number of cache lines.

7. Why the What-If Feature Matters

The project should not simply say that one configuration has a higher hit rate. It should show the relationship between a configuration change and the observed behavior.

- 4 KB, 1-way → 4 KB, 2-way: cache size stays fixed; associativity changes.
- 4 KB, 2-way → 8 KB, 2-way: associativity stays fixed; total cache capacity changes.
- Changing replacement policy: cache geometry stays the same, but eviction decisions change.
- Changing block size: the number of cache lines and address offset behavior change.

8. Suggested Python Project Structure

```
cache_analyzer/
├── main.py
├── cache.py
├── replacement/
│   ├── lru.py
│   ├── fifo.py
│   ├── lfu.py
│   ├── random.py
│   └── optimal.py
├── workload/
│   ├── sequential.py
│   ├── random.py
│   ├── loop.py
│   └── locality.py
├── simulator.py
├── metrics.py
├── visualization.py
└── database.py
```

9. Responsibilities of the Main Modules

main.py — CLI or application entry point; accepts configuration and starts experiments.

cache.py — Cache data structures, sets, ways, tags, valid bits, and block insertion.

replacement/*.py — Implement individual replacement strategies.

workload/*.py — Generate or load memory-access traces.

simulator.py — Run accesses through the cache and record each event.

metrics.py — Calculate hit/miss statistics, miss categories, evictions, and AMAT.

visualization.py — Generate charts, set heatmaps, and comparison plots.

database.py — Optionally store experiments and results for later comparison.

10. Core Cache Geometry Formulas

- Number of cache lines = cache_size / block_size
- Number of sets = number_of_cache_lines / associativity
- Block number = address // block_size
- Offset = address % block_size
- Set number = block_number % number_of_sets
- Tag = block_number // number_of_sets
- Offset bits = log2(block_size), when block size is a power of two
- Set-index bits = log2(number_of_sets), when number of sets is a power of two
- Tag bits = address_bits − offset_bits − set_index_bits

11. Example: 4 KB, 64 B, 1-Way Cache

For a 4 KB cache with 64-byte blocks and 1-way associativity:

Cache size = 4096 B

Block size = 64 B

Number of cache lines = 4096 / 64 = 64

Associativity = 1

Number of sets = 64 / 1 = 64

Therefore, each memory block maps to exactly one of the 64 sets. Because it is direct mapped, each set has only one possible cache line. A new block that maps to an occupied set replaces the block currently there; there is no choice among multiple ways.

12. Example: 256 B, 64 B, 2-Way Cache

Cache size = 256 B
Block size = 64 B
Total cache blocks = 4
Associativity = 2

Number of sets = 4 / 2 = 2

Set 0 receives blocks 0, 2, 4, 6, ...; Set 1 receives blocks 1, 3, 5, 7, ...

Each set has two ways. Therefore, Set 0 can hold two of its mapped blocks simultaneously. If a third mapped block arrives, the replacement policy decides which way to evict.

13. End-to-End Simulation Flow

8. Generate or load a memory address trace.
9. Convert each address into block number and offset.
10. Map the block to a cache set.
11. Search the set's ways for a valid matching tag.
12. If found: record a hit and update replacement-policy state.
13. If not found: record a miss and determine miss category.
14. Fetch the block from the next memory level.
15. If space exists, insert it; otherwise invoke the replacement policy.
16. Update statistics and cache state.
17. After the trace, calculate metrics and generate visualizations.
18. Optionally repeat the trace for multiple what-if configurations.

14. Example Explanation Generated by the Analyzer

Suppose the baseline is 4 KB, 1-way with a 61% hit rate and the experiment changes it to 4 KB, 4-way with an 81% hit rate. A useful explanation could state:

“Cache size and block size were unchanged. Associativity increased from 1-way to 4-way, allowing multiple blocks that map to the same set to coexist. This can reduce repeated evictions caused by set conflicts. The observed hit-rate change should therefore be examined alongside the measured conflict-miss count rather than interpreted from hit rate alone.”

This style of explanation is the core distinction between a basic cache simulator and the proposed CacheScope analyzer.

15. AMAT

The analyzer can also estimate Average Memory Access Time (AMAT). A basic model is:

AMAT = Hit Time + Miss Rate × Miss Penalty

For a multi-level cache, the model can be extended to include L1 and L2 behavior. The exact timing values should be configurable rather than hard-coded.

16. Suggested User Interface / Output

- Configuration panel: cache size, block size, associativity, replacement policy.
- Trace/workload selection: upload a trace or generate a workload.
- Live or step-by-step cache-state view.
- Summary cards: accesses, hits, misses, hit rate, miss rate, evictions, AMAT.
- Miss breakdown: compulsory vs conflict vs capacity.
- Set heatmap.
- Configuration comparison table.
- Explanation panel: 'What changed?' and 'Why did behavior change?'
- Export experiment results to CSV/JSON.

17. Important Technical Notes

- A block's set is determined by cache geometry; replacement policy does not choose the set.
- Replacement policy chooses among candidate ways in the mapped set when replacement is necessary.
- Direct-mapped cache is effectively 1-way, so there is only one possible line in each set.
- Fully associative cache has one set conceptually containing all cache lines; the placement choice is unrestricted by a set index.
- An 8 KB experiment is a hypothetical alternative configuration if the current cache is 4 KB.
- Illustrative hit rates in project documentation are examples; the simulator should calculate actual values from the supplied trace.
- Real hardware may include additional details such as write policies, prefetching, multi-level caches, virtual addressing, TLBs, coherence, and nonuniform timing. These can be future extensions.

18. Recommended Development Phases

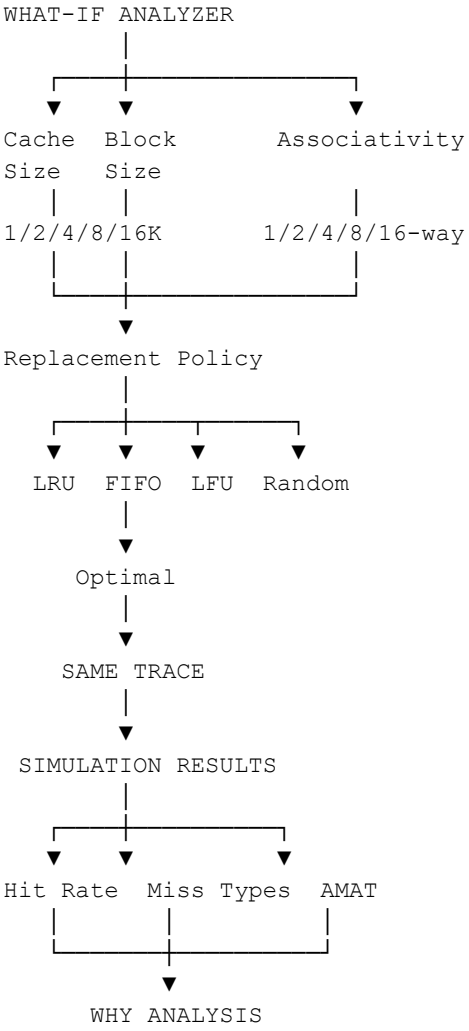
Phase	Implementation Goal
Phase 1	Implement basic cache, address-to-block/set mapping, hit/miss detection.
Phase 2	Add direct-mapped and set-associative organizations.
Phase 3	Add LRU, FIFO, Random, LFU, and Optimal replacement.
Phase 4	Add workload generators and trace loading.
Phase 5	Add miss classification and metrics.
Phase 6	Add visualization and heatmaps.
Phase 7	Add automatic what-if experiments.
Phase 8	Add explanation/report generation and optional database storage.

19. Final Project Definition

CacheScope is a cache-memory behavior analyzer rather than a simple cache simulator. Its central workflow is: give the program a workload → simulate the cache → measure behavior → change one or more cache parameters → simulate again → compare results → explain the observed differences using cache concepts such as mapping, associativity, locality, conflict misses, capacity misses, replacement, and cache capacity.

The project can therefore demonstrate both Python/software skills and computer-architecture understanding. The most important implementation goal is to make every reported result traceable to concrete cache events.

What-If Analyzer Configuration Flow



The What-If Analyzer evaluates valid combinations of cache size, block size, associativity, and replacement policy using the same memory trace. For every configuration, CacheScope recalculates the number of cache lines and sets, runs the simulation, records hit rate, miss types, AMAT, and related metrics, and explains why the results changed.